Programming Exercise 2: Linear Regression

Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

Guide #2 – Linear Regression

Introduction

The aim of this exercise is to implement the algorithm to calculate the hypothesis and get to see it working on data.

To get started with this exercise, there will be included different files that you have to complete to finish this task. Throughout the exercise, you will execute the script ***Lab2.py*** which calls functions that you will modify; please do not modify any other file, just follow the instructions of the assignment.

1. Linear regression – One variable

In this part, the idea is to implement linear regression to predict the CO^2 emission for a respective fuel consumption. Suppose you are considering a fuel consumption to calculate the amount of CO^2 emission.

The file data.txt contains the dataset for the first linear regression algorithm. The first column indicates the fuel consumption, and the second column indicates the CO^2 emission.

Complete the function ***data()*** to load the data in Python, return X and y as the data that represent the fuel consumption and the CO^2 emission respectively.

2. Plotting Data

To understand how to manipulate the data, it is often useful to visualize it. It is possible to scatter plot to visualize data, since there are just two features (2-D). Next task is to complete the ***viewData()*** function to draw the dataset samples. After running the script, the expected result should be like the following figure.

3. Computing the cost function J(w,b)

To learn how to minimize the cost function J(w,b), it Is helpful to get the monitor the convergence by computing the cost. The aim is to implement an algorithm that calculates the Cost J(w,b).

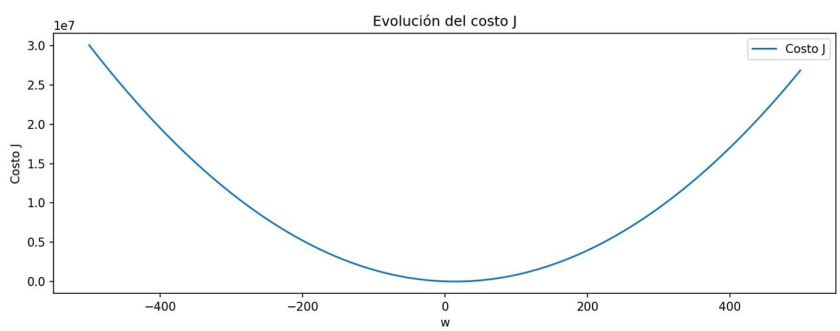
The task is to complete the function ***costFunction()*** that gets as parameters two matrices X and y. Once this algorithm is completed, in the screen would appear an expected cost of 4822.13.

Note: remember that the cost function is calculated as follows:

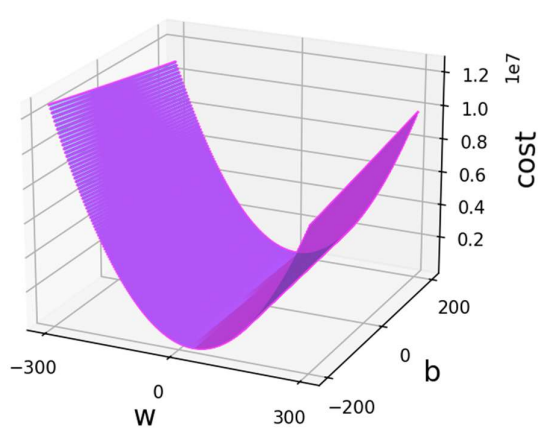
$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x_{(i)}\right) - y_{(i)}\right)^2$$

4. Generate the Cost function plot

Use the previous function to generate the cost function graph, by evaluating the cost for different values of w while it is remained a fixed b = 100.



There can be seen for example the value of the cost J when w = -200.



5. Gradient Descent algorithm

The aim of implementing gradient descent algorithm is to fit the hypothesis to the dataset. Remember that this algorithm minimizes the cost function given by the linear model, performing an iteration process that updates the value after each execution.

$$\frac{\partial}{\partial \omega} J(\omega, b) = \frac{1}{m} \sum_{i=1}^{1} \left(\omega x_{(i)} + b - y_{(i)}\right) \cdot x_{(i)}$$

$$\frac{\partial}{\partial b} J(\omega, b) = \frac{1}{m} \sum_{i=1}^{1} \left(\omega x_{(i)} + b - y_{(i)}\right)$$

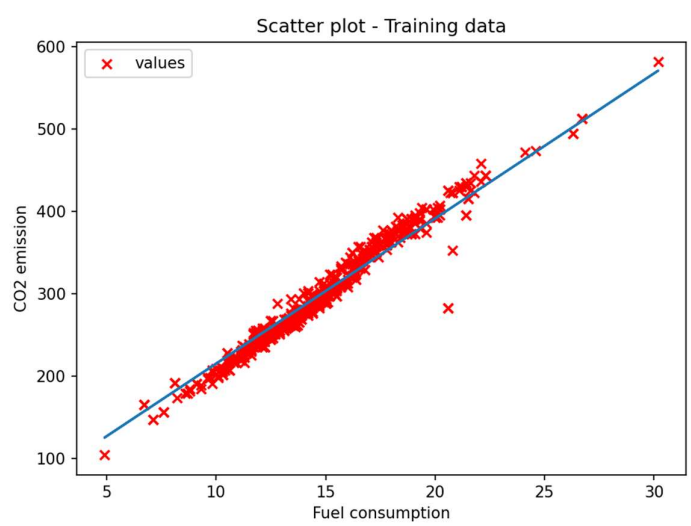
With each step of the gradient descent algorithm, parameters w and b go closer to the optimal values that get the lowest J(w,b).

In *Lab2.py,* after finishing the cost function for the linear regression model, complete the function ***gradientDescent()*** , initialized with α=0.008 and a total of 3000 iterations. After performing the algorithm correctly, the cost won't increase, and it will converge into b=38.69 and w=17.63

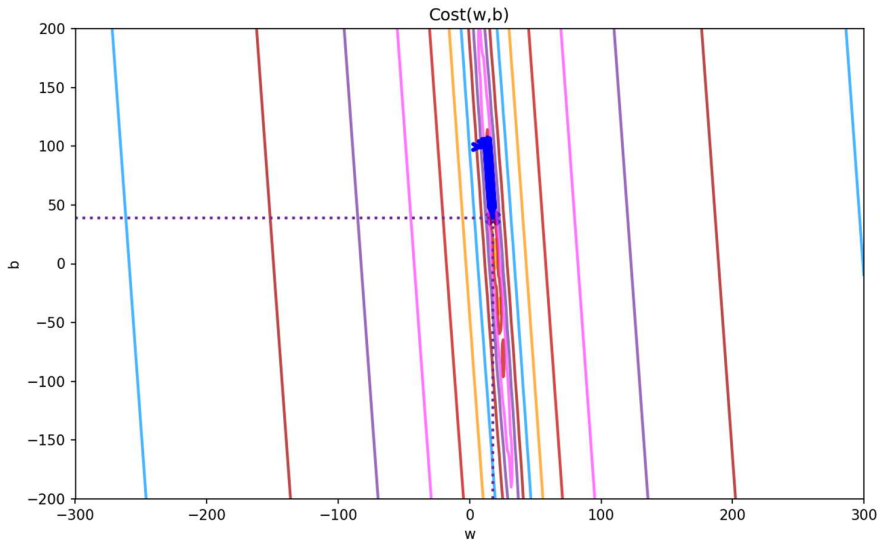
6. Plotting the linear regression

In the previous exercise, the values for w and b were found with the gradient descent algorithm.

complete the function ***linearRegression()*** by plotting the hypothesis in the figure which has all the data. The figure should be like the following:



7. Prediction and visualization

Finally, predict the CO^2 emission for a fuel consumption of 25 and 32 liters.

Once the prediction is complete, it appears a contour plot for the gradient descent algorithm and the Cost Function plot in 3D, as follows: